

VisionInspect AI

Frontend Development — Login, Image Upload & Role-Based Inspection Dashboard

Milestone 1 (Week 1 & 2) — Individual Contribution Report

Ayusha Sanjuktha

Infosys Springboard Virtual Internship Program

Table of Contents

Table of Contents.....2

1. Project Description3

2. Reference Dataset3

3. Environment Setup3

4. UI/UX Exploration & Requirement Analysis.....4

5. Design & Planning.....4

6. Proposed System (Frontend Architecture).....4

7. Implementation Pipeline.....5

8. Conclusion5

1. Project Description

VisionInspect AI is an AI-powered manufacturing defect detection and quality inspection platform being developed as part of the Infosys Springboard Virtual Internship Program (VIP). The system is designed to automatically detect product defects from images, classify defect types, assign severity scores, and generate pass/fail quality decisions for use by manufacturing and quality assurance teams.

This document covers Milestone 1 (Week 1 & 2), scoped to project initialization and core setup, and specifically reports on the frontend contribution completed independently within the assigned team. Backend services, the detection model, and infrastructure are being developed in parallel by other team members; this report focuses only on the user-facing application layer.

The frontend scope for this milestone includes an authentication/role-selection screen, a product image upload workflow, and a role-differentiated inspection dashboard for the platform's two primary user types — Quality Engineers and Factory Supervisors.

2. Reference Dataset

The detection model underlying this platform is being trained on the MVTec AD (Anomaly Detection) dataset, a standard benchmark dataset for industrial defect detection, as specified in the project brief. Model training and dataset handling fall under the backend/ML scope of the team and are outside this report.

From a frontend perspective, the relevant constraint is the set of image formats the platform must accept for inspection, matching what the dataset and detection pipeline support:

- JPG / JPEG
- PNG
- BMP
- TIFF
- WebP

The image upload component was built to accept and preview all of the above formats, so that once the detection API is connected, no frontend changes are needed to support the model's expected input types.

3. Environment Setup

The frontend was developed locally using the following environment and tooling:

- Framework: Next.js 14 (Pages Router), React 18
- Styling: Tailwind CSS 3, with a custom design token configuration (colors, fonts, background grid) defined in `tailwind.config.js`
- Language: JavaScript (JSX)
- IDE: Visual Studio Code
- Package Manager: npm
- Version Control: Git and GitHub, working on an individually assigned branch (frontend-ayusha) within the shared team repository

- Deployment: Vercel, configured to build and deploy directly from the frontend-ayusha branch for live demonstration purposes

No backend or database connection was required to set up the frontend environment at this stage, since Milestone 1 scope covers the standalone UI layer with mocked data, ahead of backend API availability.

4. UI/UX Exploration & Requirement Analysis

Before writing any code, the project specification document and system architecture diagram were reviewed to identify which parts of the platform required frontend screens, and what each user role needed to see and do.

- Reviewed the "Users & Access Channels" and "Inspection & Analytics Pipeline" sections of the architecture diagram to identify the entry points a user interacts with directly.
- Identified two primary user roles defined in the User Management Module of the specification: Quality Engineer and Factory Supervisor.
- Mapped each role to a distinct real-world responsibility — Quality Engineers run individual product inspections; Factory Supervisors monitor production-wide quality and handle escalations — which directly informed the decision to give each role a different dashboard rather than a shared, relabeled view.
- Reviewed the Severity Scoring Framework in the specification (defect size, location, defect type, and detection confidence, each weighted differently) to determine what data the dashboard needed to display per inspection.

5. Design & Planning

With requirements identified, the following design and structural decisions were made prior to implementation:

- Visual direction: an industrial "inspection console" aesthetic — dark graphite background, a technical blueprint-style grid, and a safety-orange accent colour reserved for defects/critical alerts — chosen to visually reinforce the manufacturing quality-control context rather than use a generic dashboard template.
- Typography: Space Grotesk for headings, IBM Plex Sans for body text, and IBM Plex Mono for data/scores, to distinguish narrative UI text from inspection data at a glance.
- Code structure: a clear separation between pages/ (screens), components/ (reusable UI pieces), and lib/ (data and API logic), so that backend integration later only requires changes inside lib/, not the UI components.
- Mocking strategy: since the backend authentication and detection endpoints are still under development by other team members, an isolated mock API layer was planned from the outset, so the frontend could be built, tested, and demoed end-to-end without waiting on backend completion — with a documented, one-function swap-in point for the real API once available.

6. Proposed System (Frontend Architecture)

The frontend follows a role-aware, single-page-application flow built on Next.js:

- Login Screen (pages/login.js): role selection (Quality Engineer / Factory Supervisor) and credential form; issues a temporary session token on submission pending real authentication.
- Dashboard Router (pages/dashboard.js): reads the logged-in role and conditionally renders one of two distinct views.

- Quality Engineer View: image Upload Panel with drag-and-drop, live preview, and a loading state during inspection; Result Cards showing Product Category, Defect Type, Severity Score, and Pass/Fail status; and a running Inspection Log table.
- Factory Supervisor View: plant-wide statistics (inspections today, pass rate, active lines), a 7-day trend visualization, and an Escalation Queue for reviewing and actioning critical-severity defects.
- Mock Data Layer (lib/api.js): a single runInspection() function that currently generates a severity score using the exact weighted formula from the project specification (Size 30%, Location 25%, Defect Type 25%, Confidence 20%), structured to be replaced by a real fetch() call to the /upload endpoint with no changes required elsewhere in the app.

7. Implementation Pipeline

Work was carried out across the two weeks of Milestone 1 as follows:

Week 1 — Setup & Authentication UI

- Set up the Next.js and Tailwind CSS project structure and design tokens.
- Built the login screen with role selection and form validation.
- Implemented a temporary token-based session (pending real backend auth).

Week 2 — Upload Workflow & Dashboards

- Built the drag-and-drop image upload component with file preview.
- Implemented the mock inspection API and severity scoring logic.
- Added a loading state during inspection processing.
- Built the Quality Engineer dashboard (upload, result cards, inspection log).
- Built the distinct Factory Supervisor dashboard (production overview, trend chart, escalation queue) based on mentor feedback that both roles needed visibly different interfaces.
- Verified a clean production build (npm run build) with no errors.
- Deployed the branch to Vercel for a live, shareable demo link.

Testing at this stage was manual and functional rather than automated: both role logins, the upload-to-result flow, and the escalation queue actions were each verified by hand against the live deployment before the milestone review.

8. Conclusion

Milestone 1 delivers a functional, role-aware frontend for VisionInspect AI, covering authentication UI, an image upload workflow, and two purpose-built dashboards reflecting the distinct responsibilities of Quality Engineers and Factory Supervisors. Severity scoring on the frontend currently uses a mock calculation that mirrors the project's actual scoring formula, so the interface already behaves realistically ahead of backend integration.

The clear separation between UI components and the mock data layer means the remaining integration work — connecting real authentication and the /upload detection endpoint — is isolated to a small, well-documented part of the codebase, positioning the frontend to move quickly into Milestone 2 once the backend and model endpoints are available.